PROJECT DOCUMENTATION

Project Title: Product Inventory Management

TEAM DETAILS :

>> Team ID: SWTID-2026-5714

>> Team Leader: SWETHA S – 202400044@sigc.edu

>> Team Member:

ROOBA P – 202400519@sigc.edu

RITHIKA SHREE V – 202400673@sigc.edu

SWATHY P – 202400822@sigc.edu

1.Project Overview

* Product Inventory Management System is a backend web application.

* Built using Node.js, Express.js, and MongoDB.

* RESTful APIs manage product data.

* To Provides secure login using JWT authentication.

* Supports CRUD operations for products

2. Key Objectives

* Provide secure user authentication using JWT.

* Manage product inventory through CRUD APIs.

* Insure user-specific data access.

* Build scalable backend architecture.

* Provide fast API responses with validation and error handling.

3.Inventory Management

1. An **Inventory Management System** is a system used by businesses to **track, control, and manage their products or goods**

- Instead of keeping wood in the warehouse:
- The shop **orders wood from the supplier only after receiving the order**.
- The supplier delivers the wood **just before production starts**.
- The tables are produced and delivered to the customer.
- **inventory is minimal** and storage cost is reduced.

4.TARGET USERS:

* Small business owners managing stock.

*Freelancers tracking products or services.

* Developers learning backend APIs.

* Students working on backend projects.

*Anyone needing a simple inventory backend. ⍰

5.PROJECT INSTRUCTION:

* Create backend project folder
* Configure server using Express.js
* Setup MongoDB database connection
* Implement authentication APIs
* Implement product CRUD APIs
* Test APIs using Thunder Client in VS Code

6.7PRE-REQUISITES:

Software	Purpose
Node.js	JavaScript runtime for backend

MongoDB	**Database storage**
Npm	**Package manager**
VS Code	**Development environment**

7.Knowledge Requirements:

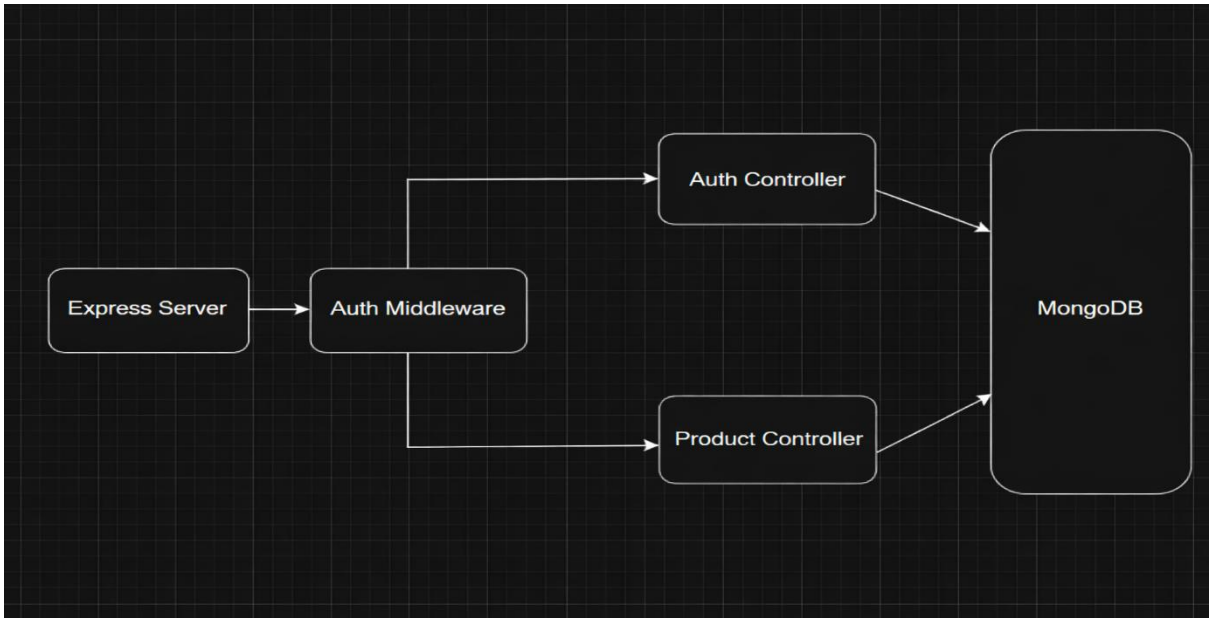
* JavaScript ES6 concepts
* Node.js and Express.js REST API development
* MongoDB and Mongoose ODM
* JWT authentication
* Middleware architecture

Project Thumbnail:

8.PROJECT ARCHITECTURE:

- Uses three-tier backend architecture.
- Node.js + Express for backend APIs.
- MongoDB used as database.
- Uses MVC pattern (Model, Controller, Routes).
- Includes validation and error handling

High-level architecture diagram illustrating the three-tier backend structure.



Node.js + Express Backend:

A **Node.js + Express backend** is a **server-side application** built using:

- **Node.js** – A JavaScript runtime that allows running JavaScript on the server.
- **Express.js** – A framework for Node.js that **simplifies building web servers and APIs**.

• RESTful API architecture

• JWT authentication middleware

• MVC pattern (Models, Controllers, Routes)

• Input validation and error handling

MongoDB Database:

MongoDB is a **NoSQL database** that stores data in **flexible, JSON-like documents** instead of traditional tables.

• Document-based NoSQL storage

• Mongoose ODM for schema modeling

• Two collections: Users and Contacts

• Indexed fields for optimized queries

9.TECHNICAL ARCHITECTURE:

Backend Architecture (Node.js + Express)

- **Controllers**: Handle authentication and product logic
- It means the **controller is responsible for processing user actions**
- **Models**: Define database schema
- It means **models describe how data is structured and stored in the database**.
- **Fields / columns** in a table
- **Data types** (string, number, date, etc.)
- **Relationships** between tables
- **Validation rules**

So, a **model acts as a blueprint for the database table**.

• **Routes**: API endpoints for authentication and products

- **Routes** define the **URLs (API endpoints) that users or clients can access to perform specific actions in the application**.
  They connect **HTTP requests** to the appropriate **controller functions**.

• **Middleware**: JWT verification and error handling

- **JWT (JSON Web Token)** middleware checks whether a user is **authenticated**.
- Using JSON Web Token, when a user logs in, the server generates a **token**.

• **Configuration**: Environment variables and database connection

1. Environment Variables:

- Environment variables store **sensitive or changeable information** outside the main code.

Examples:

- Database URL
- JWT secret key
- Port number
- API keys

2. Database Connection:

- Configuration also includes **connecting the application to the database**.

  Example using MongoDB with Mongoose:

```
const mongoose = require("mongoose");
mongoose.connect(process.env.DB_URL)
.then(() => console.log("Database connected"))
.catch((err) => console.log(err));
```

Database Architecture (MongoDB):

- Database Architecture in MongoDB refers to how data is structured, stored, and organized inside the MongoDB database.
- MongoDB follows a NoSQL document-based architecture, which means data is stored as documents in JSON-like format instead of traditional tables and rows.

Users Collection:

- _id
- name
- email
- password (hashed)
- timestamps

Products Collection:

- _id
- userId (reference to user)
- name
- price
- quantity

- description
- timestamps

Relationship: One User → Many Products

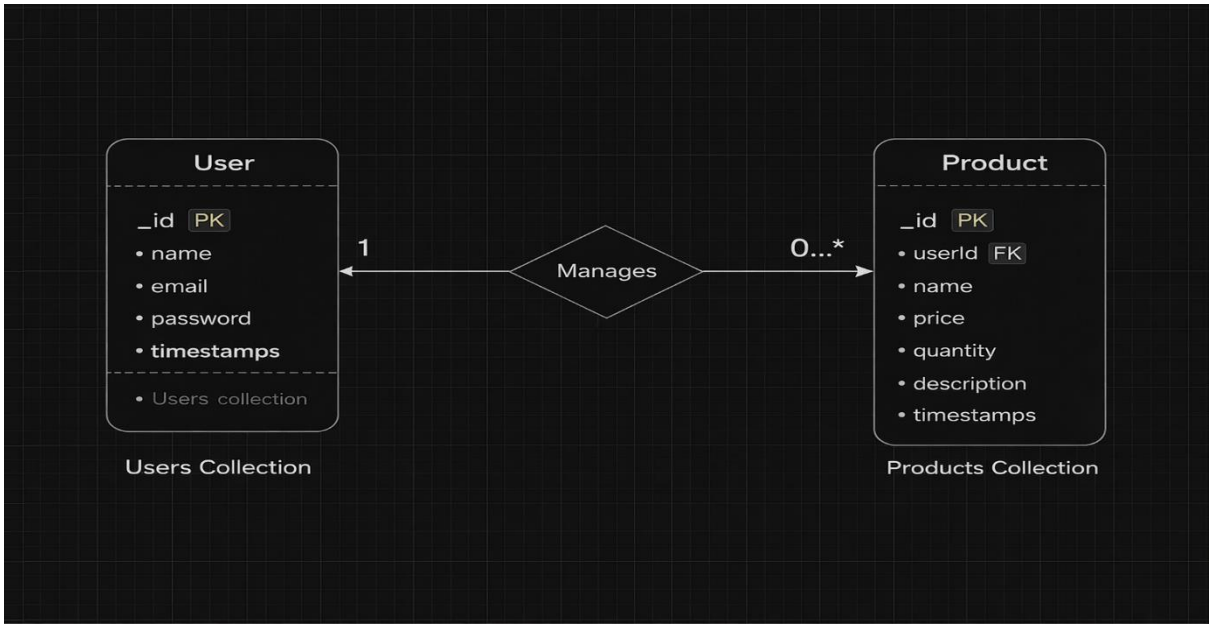
Each product belong to a specific user.

Indexes:

- email (unique)
- userId
- product name for faster queries.

ER Diagram

ER diagram showing User → Product relationship:



Features:

Core Features

S.No	Feature	Description
1	User Authentication	JWT-based register and login system
2	Product CRUD	Add, view, update and delete products

3	Private Data Access	Users access only their own products
4	Product Search	Search products by name
5	Secure API	Middleware protected routes

10.Roles and Responsibility ：

User:

Registration: Register using name, email and password

Login: Login using registered credentials

Add Product: Create product with name, price, quantity and description

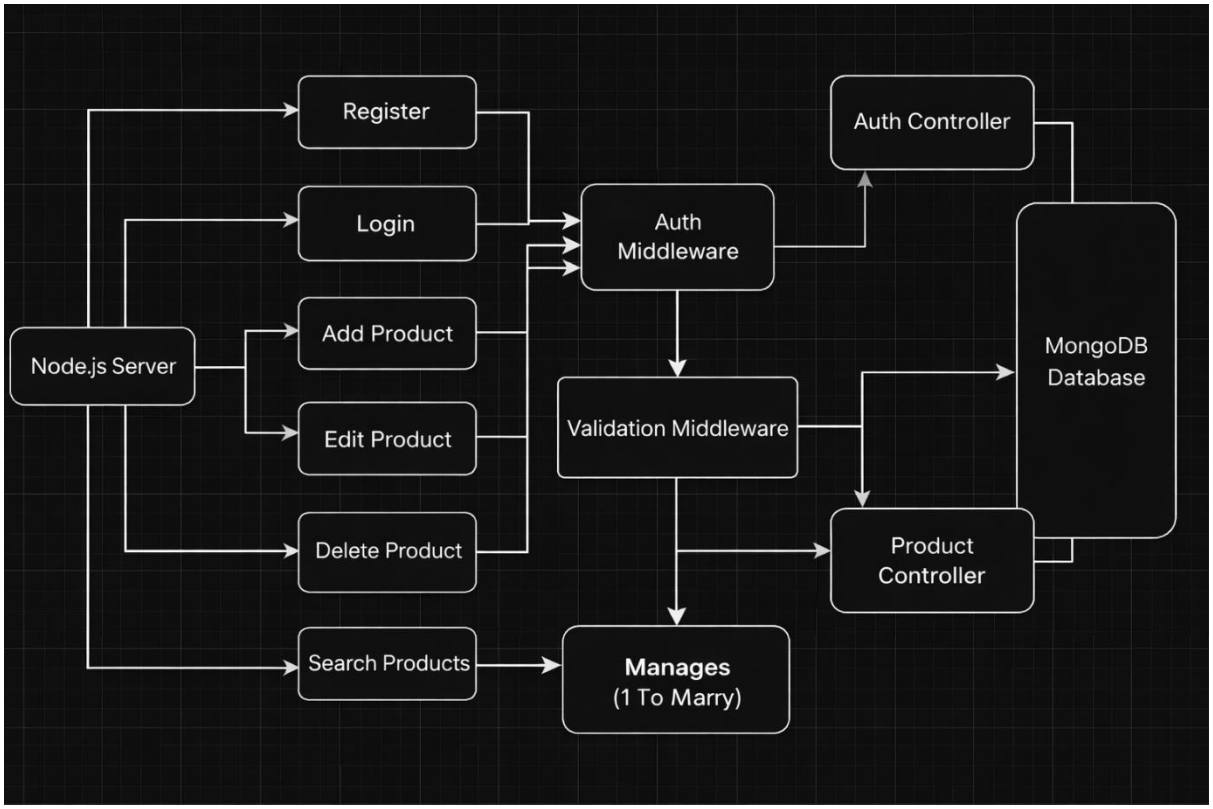
Edit Product: Update existing product

Delete Product: Remove product from inventory

Search Product: Search products using keyword

User Flow:

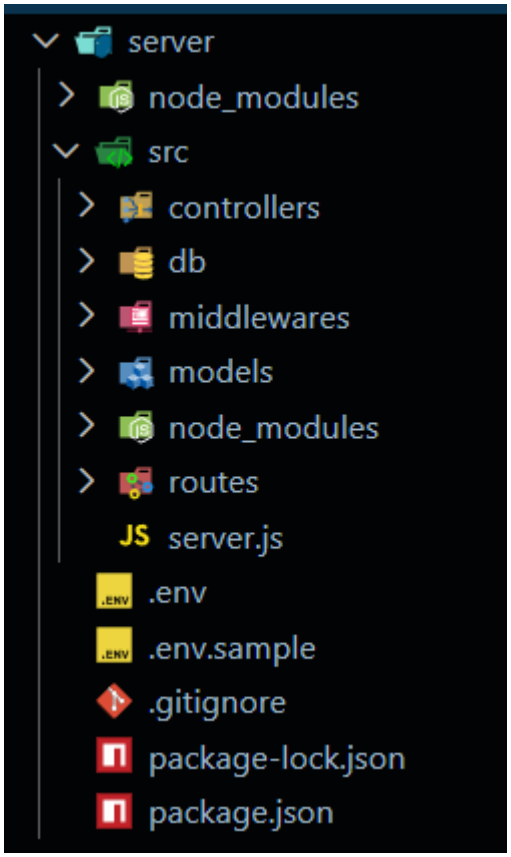
1. The user sends a register/login request.
2. The backend generates a JWT token.
3. The user sends a token in the Authorization header.
4. Middleware verifies the token.
5. The controller performs CRUD operations.
6. MongoDB stores or retrieves product data.
7. API sends response.



Project Setup and Configuration

Folder Structure:

```
server/
│
├── node_modules/
├── src/
│   ├── controllers/
│   ├── db/
│   ├── middlewares/
│   ├── models/
│   ├── routes/
│   └── server.js
│
├── .env
├── .gitignore
├── package-lock.json
└── package.json
```



Initialize Node.js Project:

Open terminal inside the server folder and run:

```
npm init -y
```

This creates the package.json file which manages project dependencies.

Install Required Packages:

Install backend dependencies:

```
npm install express mongoose cors dotenv jsonwebtoken bcryptjs
```

Package Purpose:

- express → Backend framework
- mongoose → MongoDB connection

- cors → Cross-origin requests
- dotenv → Environment variables
- jsonwebtoken → Authentication
- bcryptjs → Password hashing

Create Environment File:

Create a .env file in the root of the server folder.

Example:

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_secret_key
```

Backend Development:

Setup Express Server:

Inside src/server.js, create the basic Express server.

```
const express = require('express')
const app = express()
const port = 3000

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.listen(port, () => {
  console.log(`Example app listening on port ${port}`)
})
```

Create Controllers

Inside: src/controllers/

Create:

auth.controller.js

```
export const registerUser = async (req, res) => {
  try {···
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};


export const loginUser = async (req, res) => {
  try {···
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Auth Controllers

The Auth Controller manages all user authentication-related operations in the system. It handles user registration and login functionality while ensuring secure access using JWT tokens.

Responsibilities:

- Register new users
- Validate user input data
- Hash passwords using bcrypt before storing
- Authenticate users during login
- Generate JWT tokens for authorized access
- Send appropriate success and error responses

Main Functionalities:

- Register User: Creates a new user account after validating email uniqueness and encrypting the password.
- Login User: Verifies credentials and returns a JWT token for protected API access.

product.controller.js

```
export const createProduct = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const getProducts = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const updateProduct = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const deleteProduct = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Product Controller

- The **Product Controller** handles all product inventory operations and contains the business logic for managing products linked to individual users.

Responsibilities:

- Add new products to inventory
- Fetch products belonging to logged-in users
- Update product details like price, quantity, or description
- Delete products securely
- Perform product search functionality

Main Functionalities:

- **Create Product:** Adds a new product with a reference to the authenticated user.
- **Get Products:** Retrieves only the products created by the logged-in user.
- **Update Product:** Modifies product details while verifying ownership.

- **Delete Product:** Removes a product safely from the database.
- **Search Product:** Allows filtering products based on name.

Create Middleware

Inside: src/middlewares/

Create:auth.middleware.js

```
export const protect = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Not authorized" });
  }

  try {
    const token = authHeader.split(" ")[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    req.user = decoded;
    next();
  } catch (error) {
    res.status(401).json({ message: "Token invalid" });
  }
};
```

Auth Middleware (protect)

- The **Auth Middleware** secures protected routes by verifying the JSON Web Token (JWT) sent in the request header.
- If the token is missing or invalid, the middleware returns a **401 Unauthorized** response, preventing access to private APIs.

Create Routes

Inside: src/routes/

Create:

Auth Routes:

- Auth routes handle user authentication by providing endpoints for registration and login. These routes connect incoming requests to the Auth Controller to create new users and generate JWT tokens for secure access.

```
const authRoutes = express.Router();


authRoutes.post("/register", registerUser);

authRoutes.post("/login", loginUser);


export default authRoutes;
```

Product Routes:

- Product routes manage all inventory-related operations such as creating, retrieving, updating, and deleting products. These routes are protected using authentication middleware to ensure only authorized users can access their own product data.

```
const productRouter = express.Router();

productRouter.post("/", protect, createProduct);
productRouter.get("/", protect, getProducts);
productRouter.get("/:id", protect, getProductById);
productRouter.put("/:id", protect, updateProduct);
productRouter.delete("/:id", protect, deleteProduct);

export default productRouter;
```

Database Configuration and Setup

- Create MongoDB Atlas cluster
- Add connection string to .env
- Connect DB before API testing.

Database Connection:

Create a file inside: src/db/db.js

Purpose: Connect backend to MongoDB using mongoose.

```
import mongoose from "mongoose";

const connectDB = async () => {
    try {
        const connectionInstance = await mongoose.connect(`${process.env.MONGODB_URI}`)
        console.log(`MongoDB connected !! `);
    } catch (error) {
        console.log("MONGODB connection FAILED ", error);
        process.exit(1)
    }
}

export default connectDB
```

- The connectDB function establishes a connection between the application and MongoDB using Mongoose and environment-based configuration.
- It ensures the server starts only after a successful database connection and logs an error message if the connection fails.

Create Models:

Inside src/models/ create:

- User.js
- Product.js

User Model:

```
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
    },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
    },
    password: {
      type: String,
      required: true,
      minlength: 6,
    },
  },
  { timestamps: true }
);

export default mongoose.model("User", userSchema);
```

User Schema:

The User schema defines the structure for storing user details such as name, email, and hashed password in the database. It supports secure authentication and maintains timestamps for record tracking.

Product Model:

```
const productSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
    },

    price: {
      type: Number,
      required: true,
      min: 0,
    },

    quantity: {
      type: Number,
      required: true,
      min: 0,
      default: 0,
    },

    description: {
      type: String,
      default: "",
    },

    userId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
  },
  { timestamps: true }
);

export default mongoose.model("Product", productSchema);
```

Product Schema:

The Product schema represents inventory items and includes fields like name, price, quantity, description, and a userId reference. It ensures each product is linked to its owner, enabling private data access and management.

11.Project Execution:

Project execution means the stage of a project where the actual work is carried out to produce the project's deliverables or results. In simple terms, it is the phase where the plans are put into action.

Execution Steps

1. Open terminal in server folder.
2. Make sure you have a start script in the package.js file.
3. Run:

```
npm start
```

3. Console should display:

```
Server running on PORT
Database Connected
```

12. API Testing :

Test APIs:

Use Thunder Client to test APIs:

Register:

The register functionality allows new users to create an account by submitting their details, which are validated and securely stored with a hashed password. It ensures unique user creation before granting access to the system

POST http://localhost:5000/api/v1/auth/register

Docs Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
  "name": "User",
  "email": "user@email.com",
  "password": "123456"
}
```

Body Cookies Headers (10) Test Results 201 Created 428 ms 643 B

{} JSON Preview Visualize


```
{
    "message": "Register successful",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJpZCI6IjY5OTU5NDUzZjk2Nzc3NzlmMTViOTVhMSIsImlhdCI6MTc3MTQxMDUxNSwiZXhwIjoxNzcyMDE1MzE1fQ.
        MyYzExe75OoxKIDXGhRjR_HrLvwfjaalt4LB4pgGBDE",
    "user": {
        "id": "69959453f9677779f15b95a1",
        "name": "User",
        "email": "user@email.com"
    }
}
```


Login User:

The login functionality verifies user credentials and generates a JWT token upon successful authentication. This token is used to access protected routes securely.

POST http://localhost:5000/api/v1/auth/login

Docs Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
  "email": "user@email.com",
  "password": "123456"
}
```

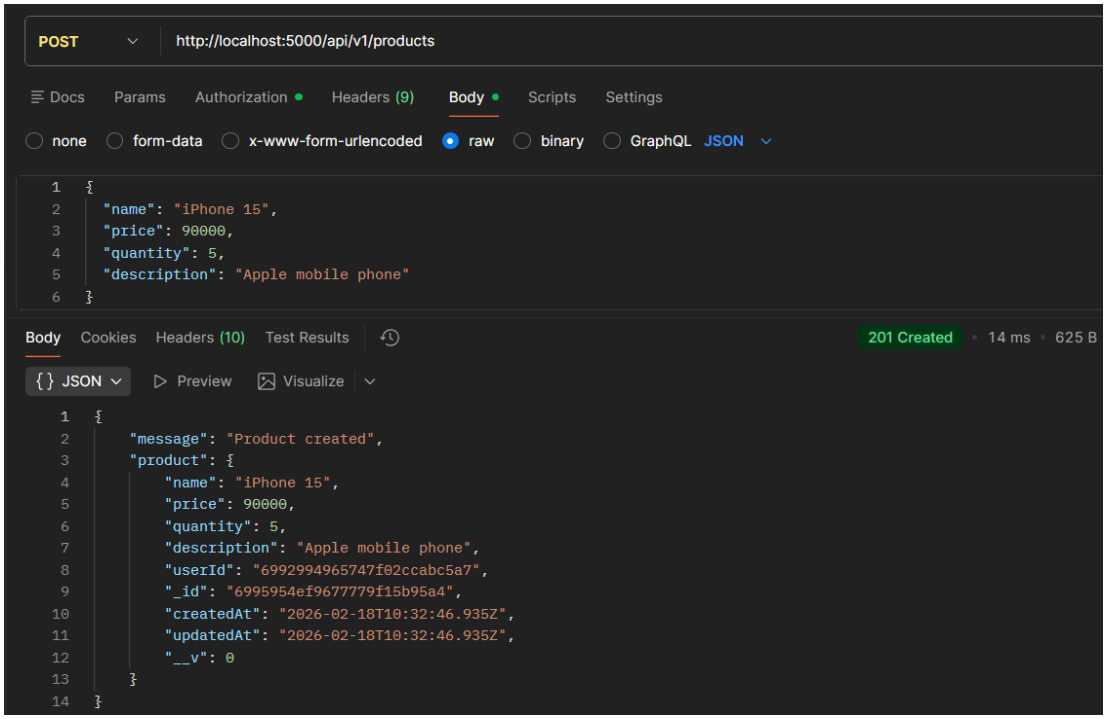
Body Cookies Headers (10) Test Results 200 OK

{} JSON Preview Visualize


```
{
    "message": "Login successful",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJpZCI6IjY5OTU5NDUzZjk2Nzc3NzlmMTViOTVhMSIsImlhdCI6MTc3MTQxMDU4NCwiZXhwIjoxNzcyMDE1Mzg0fQ.
        PyUXxz-6-4J1kgOTLBj862vTftt161srZW9AkBVoGhM",
    "user": {
        "id": "69959453f9677779f15b95a1",
        "name": "User",
        "email": "user@email.com"
    }
}
```

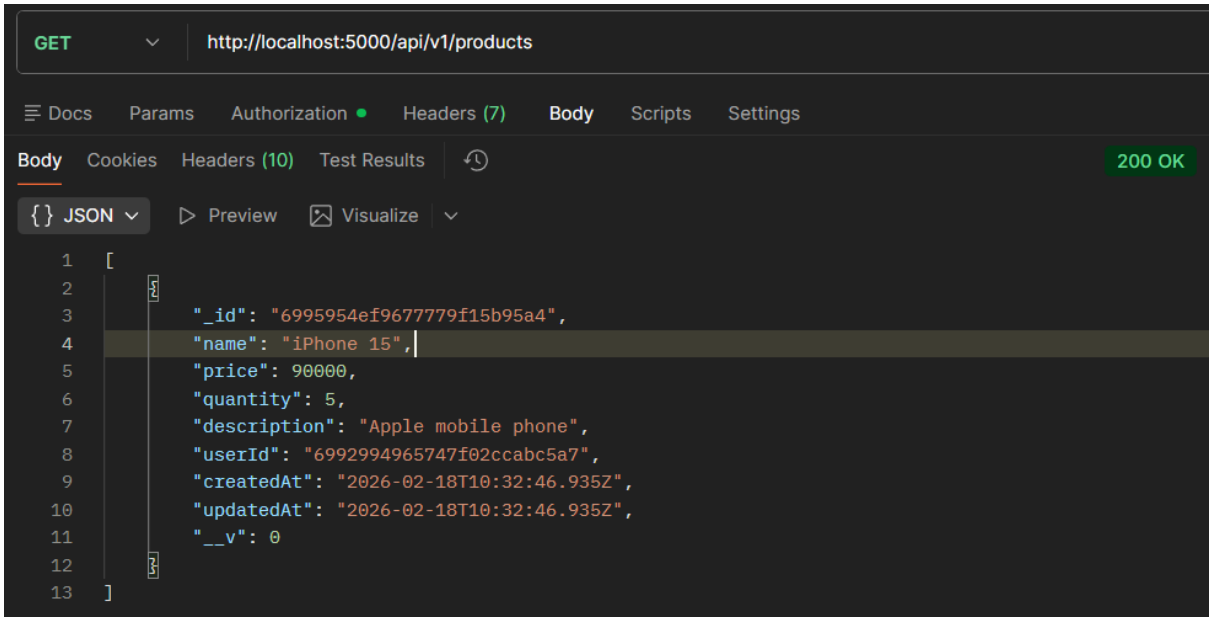

Create Product

user to add a new product to their inventory by submitting product details. The product is saved with the Allows an authenticated user's ID to maintain ownership.



Get all Products

Fetches all products that belong to the logged-in user. It ensures users can view only their own inventory data.



Update Product

Enables users to modify existing product details such as name, price, or quantity. The system verifies ownership before allowing any updates.

PUT http://localhost:5000/api/v1/products/6995954ef9677779f15b95a4

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
  "name": "iPhone 19 Pro",
  "price": 95000,
  "quantity": 10,
  "description": "Updated phone"
}
```

Body Cookies Headers (10) Test Results 200 OK 37 ms

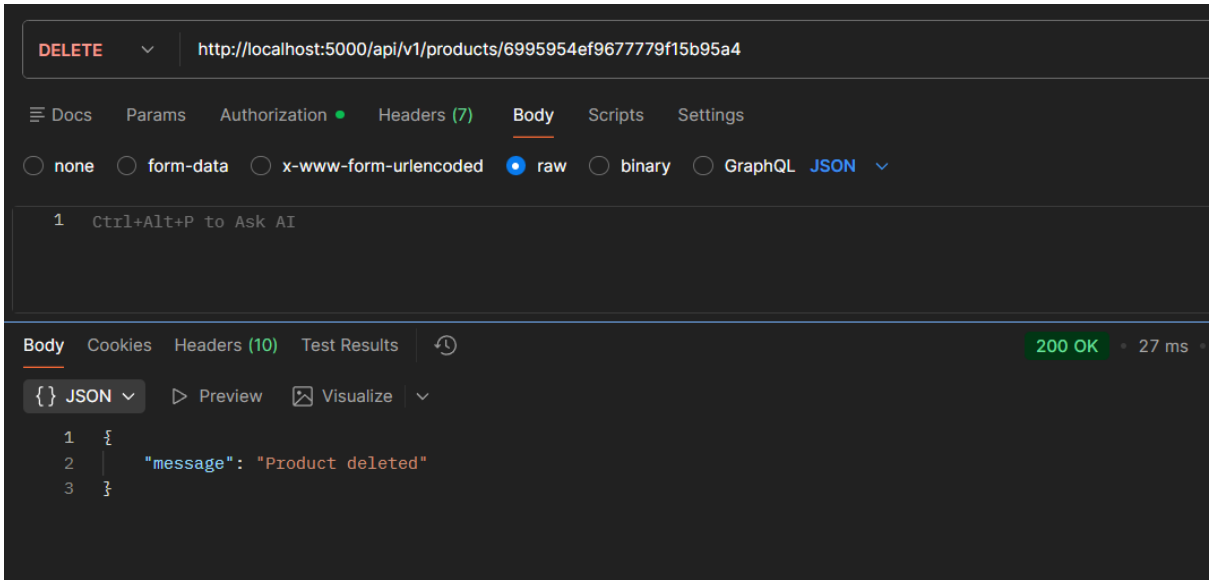
JSON Preview Visualize

```
{
    "message": "Product updated",
    "product": {
        "_id": "6995954ef9677779f15b95a4",
        "name": "iPhone 19 Pro",
        "price": 95000,
        "quantity": 10,
        "description": "Updated phone",
        "userId": "6992994965747f02ccabc5a7",
        "createdAt": "2026-02-18T10:32:46.935Z",
        "updatedAt": "2026-02-18T10:35:11.547Z",
        "__v": 0
    }
}
```

Delete Product

Allows users to permanently remove a product from their inventory. The operation is protected to ensure only the product owner can delete it.

;;;



13.Demo Link :

https://drive.google.com/file/d/10DubkMRWyxJHlb0syMLfUHPu3glQikhI/view?usp=sharing